Choose and deploy models from the model catalog in Microsoft Foundry portal

11 March 2026 11:46

1. Introduction
    a. Some common use cases for models are:
        i. Speech-to-text and text-to-speech conversion. For example, generate subtitles for videos.
        ii. Machine translation. For example, translate text from English to Japanese.
        iii. Text classification. For example, label an email as spam or not spam.
        iv. Entity extraction. For example, extract keywords or names from a document.
        v. Text summarization. For example, generate a short one-paragraph summary from a multi-page document.
        vi. Question answering. For example, provide answers to questions like "What is the capital of France?"
        vii. Reasoning. For example, solve a mathematical problem.

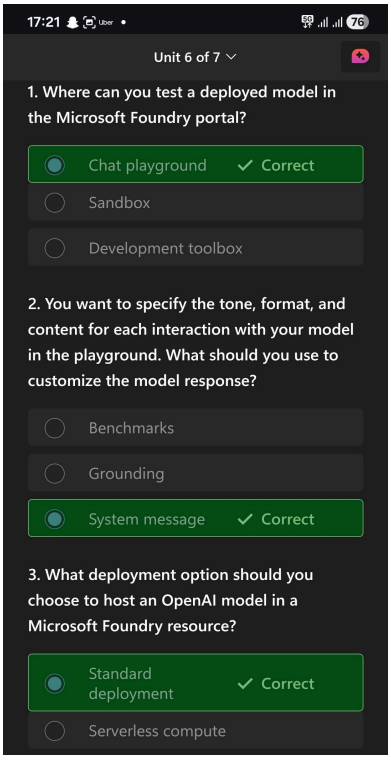
2. Explore the model catalog
    a. The model catalog in Microsoft Foundry provides a central repository of models that you can browse to find the right language model for your particular generative AI use case.
    b. LLMs like GPT-4, Mistral Large, Llama3 70B, Llama 405B, and Command R+ are powerful AI models designed for tasks that require deep reasoning, complex content generation, and extensive context understanding.
    c. SLMs like Phi3, Mistral OSS models, and Llama3 8B are efficient and cost-effective, while still handling many common Natural Language Processing (NLP) tasks. They're perfect for running on lower-end hardware or edge devices, where cost and speed are more important than model complexity.
    d. Focus on modality, task and tool while choosing LLM for your task
        i. Language models like GPT-4 and Mistral Large are also known as chat completion models, designed to generate coherent and contextually appropriate text-based responses.
        ii. When you need higher levels of performance in complex tasks like math, coding, science, strategy, and logistics, you can also use reasoning models like DeepSeek-R1 and o1.
        iii. Multimodal Models: Models like GPT-4o and Phi3-vision are capable of analyzing and generating both text and images.
        iv. If your use case involves generating images, tools like DALL·E 3 and Stability AI can create realistic visuals from text prompts.
        v. Another group of task-specific models are embedding models like Ada and Cohere. Embeddings models convert text into numerical representations and are used to improve search relevance by understanding semantic meaning. These models are often implemented in Retrieval Augmented Generation (RAG) scenarios to enhance recommendation engines by linking similar content.
        vi. When you want to build an application that interacts with other software tools dynamically, you can add function calling and JSON support. These capabilities allow AI models to work efficiently with structured data, making them useful for automating API calls, database queries, and structured data processing.
    e. Specialize with regional and domain-specific models
        i. Core42 JAIS is an Arabic language LLM, making it the best choice for applications targeting Arabic-speaking users.
        ii. Mistral Large has a strong focus on European languages, ensuring better linguistic accuracy for multilingual applications.
        iii. Nixtla TimeGEN-1 specializes in time-series forecasting, making it ideal for financial predictions, supply chain optimization, and demand forecasting.
    f. Balance flexibility and performance with open versus proprietary models
        i. Proprietary models are best for cutting-edge performance and enterprise use. Azure offers models like OpenAI's GPT-4, Mistral Large, and Cohere Command R+, which deliver industry-leading AI capabilities. These models are ideal for businesses needing enterprise-level security, support, and high accuracy.
        ii. Open-source models are best for flexibility and cost-efficiency. There are hundreds of open-source models available in the Microsoft Foundry model catalog from Hugging Face, and models from Meta, Databricks, Snowflake, and Nvidia. Open models give developers more control, allowing fine-tuning, customization, and local deployment.
        iii. Using models through the model catalog meets the key enterprise requirements for usage: Data & Privacy, Security, Compliance, Responsible AI, Content Safety
    g. When you're exploring models through the Microsoft Foundry model catalog, you can model benchmarks to compare publicly available metrics like coherence and accuracy across models and datasets.
    h. Benchmarks:
        i. Accuracy: Compares model-generated text with correct answer according to the dataset. Result is one if generated text matches the answer exactly, and zero otherwise.
        ii. Coherence: Measures whether the model output flows smoothly, reads naturally, and resembles human-like language.
        iii. Fluency: Assesses how well the generated text adheres to grammatical rules, syntactic structures, and appropriate usage of vocabulary, resulting in linguistically correct and natural-sounding responses.
        iv. Groundedness: Measures alignment between the model's generated answers and the input data.
        v. GPT Similarity: Quantifies the semantic similarity between a ground truth sentence (or document) and the prediction sentence generated by an AI model.
        vi. Quality Index: A comparative aggregate score between 0 and 1, with better-performing models scoring a higher value
        vii. Cost: The cost of using the model based on a price-per-token. Cost is a useful metric with which to compare quality, enabling you to determine an appropriate tradeoff for your needs.
    i. To evaluate how a selected model performs regarding your specific requirements, you can consider manual or automated evaluations. Manual evaluations allow you to rate your model's responses. Automated evaluations include traditional machine learning metrics and AI-assisted metrics that are calculated and generated for you.
    j. Considerations for scaling a generative AI solution include:
        i. Model deployment: Where will you deploy the model for the best balance of performance and cost
        ii. Model monitoring and optimization: How will you monitor, evaluate, and optimize model performance?Prompt management: How will you orchestrate and optimize prompts to maximize the accuracy and relevance of generated responses?Model lifecycle: How will you manage model, data, and code updates as part of an ongoing Generative AI Operations (GenAIOps) lifecycle?

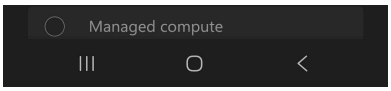
3. Deploy a model to an endpoint
    a. An endpoint is a specific URL where a deployed model or service can be accessed.
    b. Each model deployment typically has its own unique endpoint, which allows different applications to communicate with the model through an API (Application Programming Interface).
    c. When a user asks a question:
        i. An API request is sent to the endpoint.
        ii. The endpoint specifies the model that processes the request.
        iii. The result is sent back to the app through an API response.
    d. When you deploy a language model with Microsoft Foundry, you have several types available, which depend on the model you want to deploy.Deploy options include:
        i. Standard deployment: Models are hosted in the Microsoft Foundry project resource.
            1. Supported models: Microsoft Foundry models (including Azure OpenAI models and Models-as-a-service models)
            2. Hosting service: Microsoft Foundry resource
            3. Billing basis: Token-based billing
        ii. Serverless compute: Models are hosted in Microsoft-managed dedicated serverless endpoints in a Microsoft Foundry hub project.
            1. Supported models: Foundry Models with pay-as-you-go billing
            2. Hosting service: AI Project resource in a hub
            3. Billing basis: Token-based billing
        iii. Managed compute: Models are hosted in managed virtual machine images in a Microsoft Foundry hub project.
            1. Supported models: Open and custom models
            2. Hosting service: AI Project resource in a hub
            3. Billing basis: Compute-based billing

4. Optimize model performance
    a. You can carefully construct your question, or prompt, to receive better and more interesting responses.
        i. Instruct the model to act as a persona.
        ii. Guide the model to suggest better questions.
        iii. Provide a template to generate output in a specific format.
        iv. Understand how a model reasons by asking it to reflect.
        v. Add context to improve the accuracy of the model's output.
    b. As a developer, an effective approach is to add instructions to your deployed model in the form of a system prompt. The system prompt sets the model's behavior and allows you to guide the model without exposing the end user to the instructions.
    c. Prompt engineering can be an effective way to optimize model responses, but in some cases it may not provide sufficient context or guidance to always meet your exact needs. As a developer, you can consider the following additional optimization strategies to improve the relevance of your generative AI application's responses:
        i. Retrieval Augmented Generation (RAG): A technique that involves using a data source to provide grounding context to prompts. RAG can be a useful approach when you need the model to answer questions based on a specific knowledge domain or when you need the model to consider information related to events that occurred after the training data on which the model is based.
        ii. Fine-tuning: A technique that involves extending the training of a foundation model by providing example prompts and responses that reflect the desired output format and style.

5. Exercise : Explore, Deploy, and chat with language models: https://microsoftlearning.github.io/mslearn-ai-studio/Instructions/02-Explore-model-catalog.html

6. Module Assessment

7. 



8. More resources to read:
    a. https://learn.microsoft.com/en-us/azure/foundry-classic/concepts/deployments-overview